

Doubly Circular Linked List

1.Insertion at beginning

main.c	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 4 struct node{ 5 int data; 6 struct node *prev; 7 struct node *next; 8 }; 9 10 struct node *head = NULL; 11 12 void beginning() { 13 struct node * first,*second,*tail; 14 15 first = (struct node*)malloc(sizeof(struct node)); 16 second = (struct node*)malloc(sizeof(struct node)); 17 tail = (struct node*)malloc(sizeof(struct node)); 18 19 first->data = 20; 20 second->data = 30; 21 tail->data = 40; 22 23 head = first; 24 25 first->next = second; 26 second->prev = first; 27</pre>	<pre>List: 10 20 30 40 === Code Execution Successful ===</pre>
<pre>28 second->next =tail; 29 tail->prev = second; 30 31 tail->next = head; 32 head->prev = tail; 33 } 34 35 void insertbegin(int data){ 36 struct node *newnode,*tail; 37 38 newnode = (struct node*)malloc(sizeof(struct node)); 39 newnode->data = data; 40 41 tail = head->prev; 42 43 newnode->next = head; 44 newnode->prev =tail; 45 46 tail->next = newnode; 47 head->prev = newnode; 48 49 head = newnode; 50 } 51 52 void display(){ 53 struct node *temp = head; 54</pre>	<pre>List: 10 20 30 40 === Code Execution Successful ===</pre>

```
50 }
51
52 - void display(){
53     struct node *temp = head;
54
55 -     if(head == NULL){
56         printf("List is empty");
57         return;
58     }
59
60     printf("%d ", temp->data);
61     temp = temp->next;
62
63 -     while(temp != head){
64         printf("%d ", temp->data);
65         temp = temp->next;
66     }
67 }
68
69 - int main(){
70     beginning();
71     insertbegin(10);
72
73     printf("List: ");
74     display();
75
76     return 0;
77 }
```

List: 10 20 30 40

=== Code Execution Successful ===

2. Insertion at End

main.c	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 4 struct node { 5 int data; 6 struct node *prev, *next; 7 }; 8 9 struct node *head = NULL; 10 11 void display() { 12 struct node *temp = head; 13 while(temp != NULL) { 14 printf("%d <-> ", temp->data); 15 temp = temp->next; 16 } 17 printf("NULL\n"); 18 } 19 20 void insertatEnd(int value) { 21 struct node *newnode, *temp; 22 23 newnode = (struct node*)malloc(sizeof(struct node)); 24 newnode->data = value; 25 newnode->next = NULL; 26 27 if(head == NULL) {</pre>	<pre>Before insertion: 10 <-> 20 <-> NULL After insertion: 10 <-> 20 <-> 30 <-> NULL === Code Execution Successful ===</pre>
<pre>27 if(head == NULL) { 28 newnode->prev = NULL; 29 head = newnode; 30 return; 31 } 32 33 temp = head; 34 while(temp->next != NULL) { 35 temp = temp->next; 36 } 37 38 temp->next = newnode; 39 newnode->prev = temp; 40 } 41 42 int main() { 43 44 insertatEnd(10); 45 insertatEnd(20); 46 47 printf("Before insertion: "); 48 display(); 49 50 insertatEnd(30); 51 printf("After insertion: "); 52 display(); 53 return 0; 54 }</pre>	<pre>Before insertion: 10 <-> 20 <-> NULL After insertion: 10 <-> 20 <-> 30 <-> NULL === Code Execution Successful ===</pre>

3.Insertion at Position.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node {
5      int data;
6      struct node *prev, *next;
7  };
8
9  struct node *head = NULL;
10
11 void insertatEnd(int value) {
12     struct node *newnode, *temp;
13
14     newnode = (struct node*)malloc(sizeof(struct node));
15     newnode->data = value;
16     newnode->next = NULL;
17
18     if(head == NULL) {
19         newnode->prev = NULL;
20         head = newnode;
21         return;
22     }
23
24     temp = head;
25     while(temp->next != NULL)
26         temp = temp->next;
27
```

```
10 <-> 20 <-> 40 <-> NULL
10 <-> 20 <-> 30 <-> 40 <-> NULL
```

=== Code Execution Successful ===

```
28     temp->next = newnode;
29     newnode->prev = temp;
30 }
31
32 void insertatPosition(int value, int pos) {
33     struct node *newnode, *temp;
34     int i;
35
36     newnode = (struct node*)malloc(sizeof(struct node));
37     newnode->data = value;
38
39     temp = head;
40
41     for(i = 1; i < pos-1; i++) {
42         temp = temp->next;
43     }
44
45     newnode->next = temp->next;
46     newnode->prev = temp;
47
48     if(temp->next != NULL)
49         temp->next->prev = newnode;
50
51     temp->next = newnode;
52 }
53
54 void display() {
```

```
10 <-> 20 <-> 40 <-> NULL
10 <-> 20 <-> 30 <-> 40 <-> NULL
```

=== Code Execution Successful ===

```
54 void display() {
55     struct node *temp = head;
56     while(temp != NULL) {
57         printf("%d <-> ", temp->data);
58         temp = temp->next;
59     }
60     printf("NULL\n");
61 }
62
63 int main() {
64
65     insertatEnd(10);
66     insertatEnd(20);
67     insertatEnd(40);
68
69     display();
70     insertatPosition(30, 3);
71     display();
72     return 0;
73 }
```

```
10 <-> 20 <-> 40 <-> NULL
10 <-> 20 <-> 30 <-> 40 <-> NULL
```

```
=== Code Execution Successful ===
```

4. Deletion at Beginning.

main.c	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 4 struct node { 5 int data; 6 struct node *prev, *next; 7 }; 8 9 struct node *head = NULL; 10 11 void insertatEnd(int value) { 12 struct node *newnode, *temp; 13 14 newnode = (struct node*)malloc(sizeof(struct node)); 15 newnode->data = value; 16 newnode->next = NULL; 17 18 if(head == NULL) { 19 newnode->prev = NULL; 20 head = newnode; 21 return; 22 } 23 24 temp = head; 25 while(temp->next != NULL) { 26 temp = temp->next; 27 }</pre>	<pre>Before deletion: 100 <-> 200 <-> 300 <-> NULL After deletion: 200 <-> 300 <-> NULL === Code Execution Successful ===</pre>

<pre>29 temp->next = newnode; 30 newnode->prev = temp; 31 } 32 33 void deleteatBeginning() { 34 struct node *temp; 35 36 if(head == NULL) { 37 printf("List is empty\n"); 38 return; 39 } 40 41 temp = head; 42 head = head->next; 43 44 if(head != NULL) 45 head->prev = NULL; 46 47 free(temp); 48 } 49 50 void display() { 51 struct node *temp = head; 52 while(temp != NULL) { 53 printf("%d <-> ", temp->data); 54 temp = temp->next; 55 }</pre>	<pre>Before deletion: 100 <-> 200 <-> 300 <-> NULL After deletion: 200 <-> 300 <-> NULL === Code Execution Successful ===</pre>
--	--

```

41     temp = head;
42     head = head->next;
43
44     if(head != NULL)
45         head->prev = NULL;
46
47     free(temp);
48 }
49
50 void display() {
51     struct node *temp = head;
52     while(temp != NULL) {
53         printf("%d <-> ", temp->data);
54         temp = temp->next;
55     }
56     printf("NULL\n");
57 }
58 int main() {
59     insertatEnd(100);
60     insertatEnd(200);
61     insertatEnd(300);
62     printf("Before deletion: ");
63     display();
64     deleteatBeginning();
65     printf("After deletion: ");
66     display();
67     return 0;
68 }

```

^ Before deletion: 100 <-> 200 <-> 300 <-> NULL
 After deletion: 200 <-> 300 <-> NULL

=== Code Execution Successful ===

5. Deletion at End

main.c	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 4 struct node{ 5 int data; 6 struct node *prev; 7 struct node *next; 8 }; 9 10 struct node *head = NULL; 11 12 void end(){ 13 struct node *first,*second; 14 15 head = (struct node*)malloc(sizeof(struct node)); 16 first = (struct node*)malloc(sizeof(struct node)); 17 second = (struct node*)malloc(sizeof(struct node)); 18 19 head->data = 65; 20 first->data = 115; 21 second->data = 30; 22 23 head->prev = NULL; 24 head->next = first; 25 26 first->prev = head; 27 first->next = second;</pre>	<pre>Before deletion: 65 115 30 After deletion: 65 115 === Code Execution Successful ===</pre>
<pre>29 second->prev = first; 30 second->next = NULL; 31 } 32 33 void delete_end(){ 34 struct node *temp = head; 35 36 if(head == NULL){ 37 printf("List is empty\n"); 38 return; 39 } 40 41 while(temp->next != NULL){ 42 temp = temp->next; 43 } 44 45 temp->prev->next = NULL; 46 free(temp); 47 } 48 49 void display(){ 50 struct node *temp = head; 51 52 while(temp != NULL){ 53 printf("%d ", temp->data); 54 temp = temp->next; 55 }</pre>	<pre>Before deletion: 65 115 30 After deletion: 65 115 === Code Execution Successful ===</pre>


```
44     temp->prev->next = NULL;
45     free(temp);
46 }
47
48
49 void display(){
50     struct node *temp = head;
51
52     while(temp != NULL){
53         printf("%d ", temp->data);
54         temp = temp->next;
55     }
56 }
57
58 int main(){
59     end();
60
61     printf("Before deletion: ");
62     display();
63
64     delete_end();
65
66     printf("\nAfter deletion: ");
67     display();
68
69     return 0;
70 }
```

Before deletion: 65 115 30
After deletion: 65 115

=== Code Execution Successful ===

6. Deletion at Position.

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node{
5     int data;
6     struct node *prev;
7     struct node *next;
8 };
9
10 struct node *head = NULL;
11
12 void create(){
13     struct node *first, *second;
14
15     head = (struct node*)malloc(sizeof(struct node));
16     first = (struct node*)malloc(sizeof(struct node));
17     second = (struct node*)malloc(sizeof(struct node));
18
19     head->data = 10;
20     first->data = 20;
```

Before deletion: 10 20 30
After deletion: 10 30

=== Code Execution Successful ===

```

20     first->data = 20;
21     second->data = 30;
22
23     head->prev = NULL;
24     head->next = first;
25
26     first->prev = head;
27     first->next = second;
28
29     second->prev = first;
30     second->next = NULL;
31 }
32
33 void delete_pos(int pos){
34     struct node *temp = head;
35     int i;
36
37     if(head == NULL){
38         printf("List is empty\n");
39         return;

```

```

^ Before deletion: 10 20 30
After deletion: 10 30

=== Code Execution Successful ===

```

```

39         return;
40     }
41
42     if(pos == 1){
43         head = head->next;
44         if(head != NULL)
45             head->prev = NULL;
46         free(temp);
47         return;
48     }
49
50     for(i = 1; i < pos-1; i++){
51         temp = temp->next;
52     }
53
54     struct node *del = temp->next;
55
56     temp->next = del->next;
57
58     if(del->next != NULL)
59         del->next->prev = temp;
60
61     free(del);
62 }
63
64 void display(){
65     struct node *temp = head;

```

```

^ Before deletion: 10 20 30
After deletion: 10 30

=== Code Execution Successful ===

```

```

59     del->next->prev = temp;
60
61     free(del);
62 }
63
64 void display(){
65     struct node *temp = head;
66
67     while(temp != NULL){
68         printf("%d ", temp->data);
69         temp = temp->next;
70     }
71 }
72
73 int main(){
74
75     create();
76
77     printf("Before deletion: ");
78     display();
79
80     delete_pos(2);
81
82     printf("\nAfter deletion: ");
83     display();
84
85     return 0;
86 }

```

^ Before deletion: 10 20 30
 After deletion: 10 30

=== Code Execution Successful ===